A Verification Framework for Distributed Runtime Verification of Linearizability

Abstract

Linearizability is the standard correctness condition for concurrent data structures. In practice, verifying a single execution is NP-complete, and runtime verification tools often interfere with the actual execution.

This paper presents an open-source verification framework for distributed runtime verification of linearizability. The framework treats the system as a black box, requiring only the class name of the concurrent implementation. The framework has two layers: an instrumentation layer and a monitoring layer. The instrumentation layer provides two distributed algorithms based on non-linearizable snapshot objects. These algorithms obtain the current execution asynchronously, without modifying the system under inspection. In the monitoring layer, if the observed execution is not linearizable, the framework guarantees that the real execution is also non-linearizable. When the execution is linearizable, the framework produces a witness using a linearizability checker. The framework is implemented in Java, with snapshot components written in Clojure. It currently supports concurrent data structures such as sets, maps, queues, and deques, and can be extended to other data structures by providing their sequential specification. The software is publicly available and supports reproducible experiments.

Keywords: Concurrent data structures, Distributed instrumentation, Linearizability, Runtime verification, Software tool, Verification framework

1. Introduction

Runtime Verification (RV) verifies correctness properties of the current execution of a system. In this work, we focus on verifying linearizability of a single execution at runtime. In concurrent algorithms, operations may overlap, so executions are naturally described as sequences of invocation and response events that form a partial order [4].

Runtime verification of linearizability involves two main tasks. The first task is to detect the current execution of the system. The second task is to decide whether this execution is linearizable.

Deciding whether a single execution is linearizable is NP-complete [3]. Nevertheless, several checking techniques have been proposed to construct a linearization when one exists, such as the *Just-In-Time (JIT)* technique introduced in [5].

Detecting the execution is also challenging. Existing RV systems rely on intrusive instrumentation techniques, such as bytecode weaving (e.g., AspectJ). These techniques can alter the detected execution and introduce false positives and false negatives, as shown experimentally by Falcone and El-Hoyakem in [2].

Contribution

We present an open-source verification framework for distributed runtime verification of linearizability. The framework is based on the distributed instrumentation algorithms introduced in [6], that use non-linearizable snapshot objects (collect objects) to obtain the current execution without interfering with the system under inspection.

Our framework guarantees that if the collected execution is not linearizable, then the real execution is also non-linearizable, thus avoiding the behaviors introduced by intrusive instrumentation [2]. When the execution is linearizable, the framework produces a witness using a JIT-based checker developed in [5].

The framework supports common concurrent data structures such as queues, sets, maps, and deques, and is designed to be easily extensible and reproducible.

2. Software Overview

The software consists of a high-level verification framework (Figure 2) and a low-level system architecture (Figure 1). In the low-level system, the Executioner runs a concurrent implementation under inspection, transformed into a `DistAlgorithm`. The Wrapper invokes its methods while constructing a partial order of operations to describe the current execution, and the Verifier obtains this current execution to decide linearizability.

2.1. System Architecture

As shown in Figure 1, the framework consists of the following components.

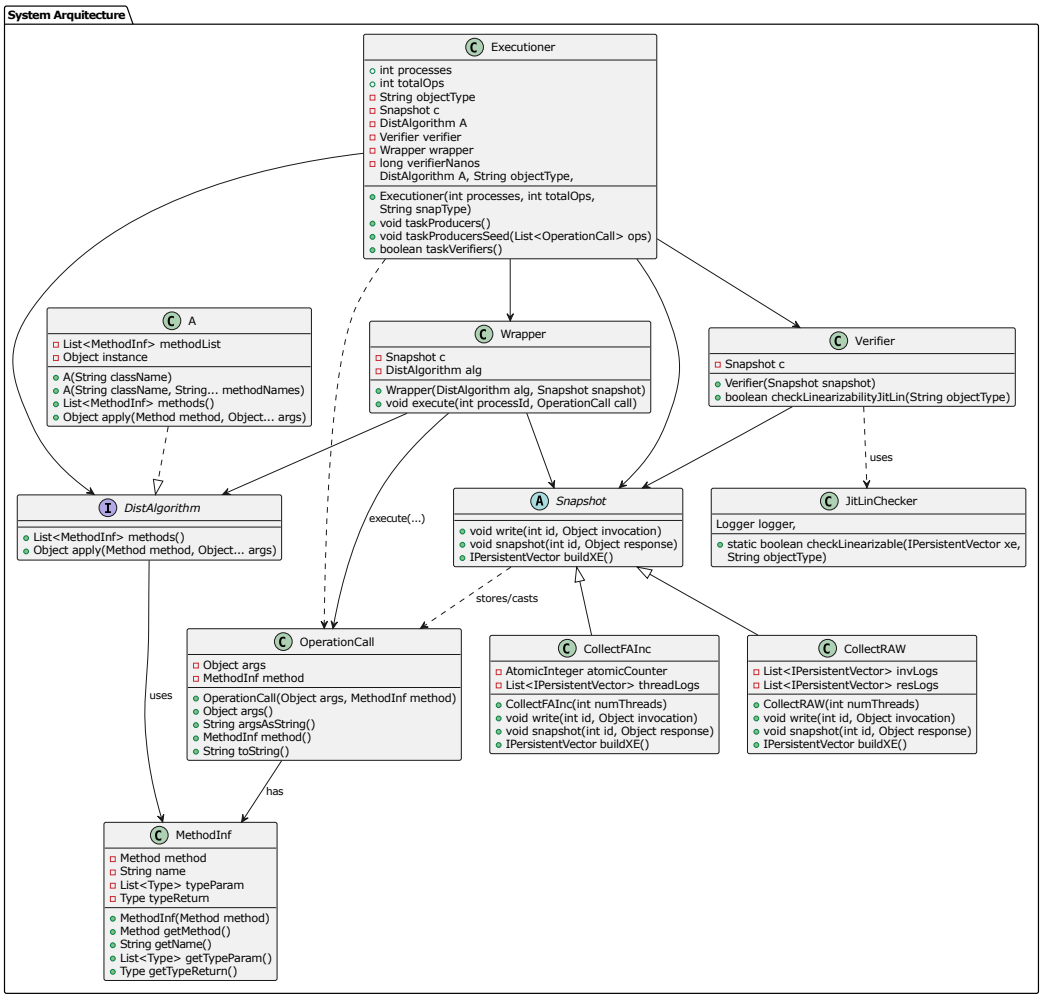

Figure 1: System architecture of the verification framework.

DistAlgorithm. This component is an abstraction layer for the concurrent implementation under inspection. It saves the set of operations and a generic `apply` method, allowing the framework to handle different data structures uniformly. Existing Java concurrent classes are wrapped using reflection.

Executioner. This component drives the execution of the experiment. It instantiates the Snapshot, Wrapper, and Verifier components, spawns worker threads, and collects the final verdict and execution time.

Wrapper. This component acts as the instrumentation layer, introduced in [1]. For each operation, it records an invocation event, executes the operation on the underlying object, and records the corresponding response event in the Snapshot object (see Algorithm 1).

Algorithm 1 Wrapper: distributed instrumentation mechanism

```
procedure EXECUTE(op, params)
    id ← ThreadId()
    C.WRITE(id, ⟨invoke, op, params⟩)
    res ← A.APPLY(op, params)
    C.SNAPSHOT(id, ⟨return, res⟩)
end procedure
```

Snapshot objects. The framework provides two non-linearizable snapshot implementations (collect implementations): The CollectFAInc implementation induces a partial order using atomic fetch-and-increment, and the CollectRAW implementation preserves per-thread order and reconstructs precedence relations from snapshot views. Both expose the same interface and can be used interchangeably.

Verifier. The Verifier shares the Snapshot object with the Wrapper. After execution, it reconstructs a *well-formed* execution X_E and invokes a JIT-based linearizability checker to produce a verdict and, when applicable, a linearization witness.

2.2. Verification Framework

The verification framework provides a configurable execution environment to run experiments with different workloads, thread counts, and snapshot implementations. The users can select the concurrent object under test, the instrumentation strategy, and the verification parameters without modifying the original implementation.

Figure 2 summarizes the interaction between the Executioner, Wrapper, Snapshot objects, and the Verifier during an execution.

3. Reproducibility and Usage

The framework is implemented in Java and Clojure and distributed as open-source software. It can be built using Maven and executed via command-line scripts. The public repository includes detailed documentation, example workloads, configuration files, and scripts that allow users and reviewers to reproduce the experiments reported in this paper.

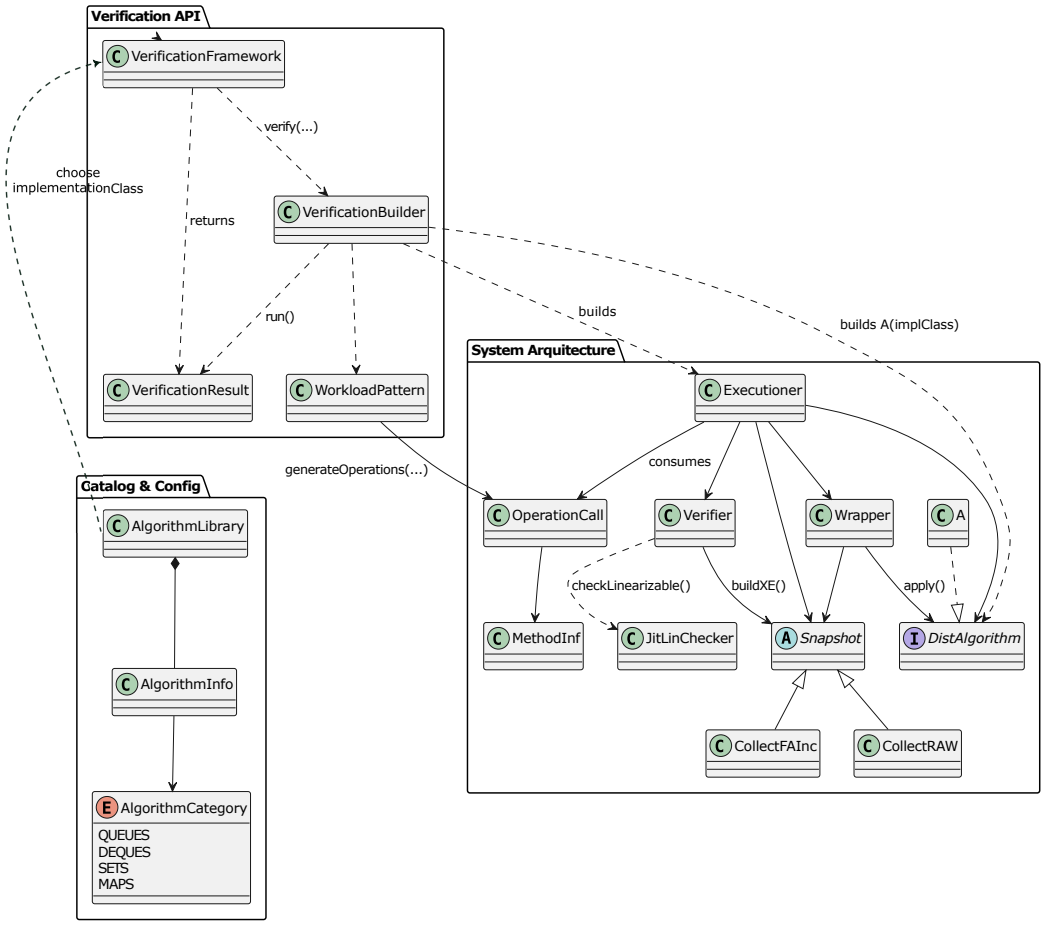

Figure 2: Verification Framework

4. Availability

The software is publicly available under an open-source license. The repository provides the complete source code, user documentation, developer notes, and usage examples. Additional material supporting reproducibility is included as part of the software distribution.

Software Metadata

Item	Description
Software name	Distributed Runtime Verification Framework of Linearizability
Version	v1.0.0
Release date	2025-12-15
Authors	Anonymous (to be filled after review)
Repository (public)	`https://github.com/PRISM-Concurrent/efficient-distributed-rv`
Persistent identifier (DOI)	To be assigned
License	Apache License, Version 2.0 (Apache-2.0)
Programming language	Java, Clojure
Build system	Maven
Operating systems	Linux, macOS, Windows
Requirements	JDK 21+, Maven 3.9+, Clojure runtime
Documentation	`README.md`, `INSTALL.md`, `API_USAGE_GUIDE.org`, `USER_MANUAL.md`, `API_EXAMPLES.md`, `QUICK_START.md`
Support email	Available after acceptance

Table 1: Software metadata required for an Original Software Publication.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work, the authors used ChatGPT in order to assist with language editing, grammar, style revision, and LaTeXformatting of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Castañeda, A., Rodríguez, G.V., 2025. Asynchronous wait-free runtime verification and enforcement of linearizability. J. ACM URL: `https://doi.org/10.1145/3777409`, doi:`10.1145/3777409`. just Accepted.

[2] El-Hokayem, A., Falcone, Y., 2018. Can we monitor all multi-threaded programs?, in: Colombo, C., Leucker, M. (Eds.), Runtime Verification, Springer International Publishing, Cham. pp. 64–89. URL: https://doi.org/10.1007/978-3-030-03769-7_6, doi:10.1007/978-3-030-03769-7_6.

[3] Gibbons, P.B., Korach, E., 1997. Testing shared memories. SIAM Journal on Computing 26, 1208–1244. doi:10.1137/S0097539794279614.

[4] Herlihy, M.P., Wing, J.M., 1990. Linearizability: a correctness condition for concurrent objects. ACM Trans. Program. Lang. Syst. 12, 463–492. URL: https://doi.org/10.1145/78969.78972, doi:10.1145/78969.78972.

[5] Lowe, G., 2017. Testing for linearizability. Concurrency and Computation: Practice and Experience 29, e3928. doi:https://doi.org/10.1002/cpe.3928. e3928 cpe.3928.

[6] Rodríguez, G.V., Castañeda, A., 2024. Towards efficient runtime verified linearizable algorithms, in: Ábrahám, E., Abbas, H. (Eds.), Runtime Verification, Springer Nature Switzerland, Cham. pp. 262–281. URL: https://doi.org/10.1007/978-3-031-74234-7_17, doi:10.1007/978-3-031-74234-7_17.